

# CORRIGÉ DÉTAILLÉ — TD N14

## Algorithmique

Première générale — spécialité mathématiques

**Comment utiliser ce corrigé ?** Pour lire un programme, on suit les variables dans l'ordre des instructions. Pour une boucle, on repère l'initialisation, la condition ou le nombre de passages, la mise à jour des variables et la valeur finalement affichée.

**POINT FONDAMENTAL À RETENIR EN MATHÉMATIQUES** En Python,

`=` réalise une affectation, `==` teste une égalité.

Une boucle `for` effectue un nombre de passages fixé par l'itérable. Une boucle `while` se poursuit tant que sa condition est vraie : une variable intervenant dans cette condition doit donc être modifiée pour permettre l'arrêt.

### A — Réactivation : lire et exécuter

**Exercice 1**

REP

**Énoncé** Associer chaque instruction Python à son rôle.

|                        |  |
|------------------------|--|
| <code>x = 4</code>     |  |
| <code>print(x)</code>  |  |
| <code>x == 4</code>    |  |
| <code>x = x + 1</code> |  |
| <code>input()</code>   |  |

Rôles possibles : afficher, affecter, tester une égalité, saisir, augmenter.

**Méthode** On associe chaque instruction à l'action qu'elle réalise sur une variable ou sur l'affichage.

|                        |                            |
|------------------------|----------------------------|
| <code>x = 4</code>     | affecter la valeur 4 à $x$ |
| <code>print(x)</code>  | afficher la valeur de $x$  |
| <code>x == 4</code>    | tester si $x$ est égal à 4 |
| <code>x = x + 1</code> | augmenter $x$ de 1         |
| <code>input()</code>   | saisir une valeur          |

**Conclusion.** Affecter ; afficher ; tester une égalité ; augmenter ; saisir.

**Exercice 2**

CAL, REP

**Énoncé** Faire tourner le programme à la main.

```
1 a = 5
2 b = 2
3 a = a + b
4 b = 3*a
5 print(a)
6 print(b)
```

Quelles valeurs sont affichées ?

**Méthode** On exécute les instructions dans l'ordre et on remplace la valeur d'une variable dès qu'une nouvelle affectation est effectuée.

Au départ,

$$a = 5, \quad b = 2.$$

L'instruction

$$a = a + b$$

donne

$$a = 5 + 2 = 7.$$

À ce moment-là,

$$a = 7, \quad b = 2.$$

Puis

$$b = 3*a$$

donne

$$b = 3 \times 7 = 21.$$

Le programme affiche donc successivement

$$7 \quad \text{puis} \quad 21.$$

**Conclusion.** Les valeurs affichées sont 7 et 21.

**Exercice 3**

CAL

**Énoncé** On considère le programme suivant.

```
1 u = 7
2 u = 2*u - 3
3 u = 2*u - 3
4 print(u)
```

Compléter :

$$u \leftarrow 7 \longrightarrow \longrightarrow$$

Quelle valeur est affichée ?

**Méthode** La même instruction est appliquée deux fois. On utilise à chaque fois la valeur courante de  $u$ .

Au départ,

$$u = 7.$$

Premier passage :

$$u = 2 \times 7 - 3 = 11.$$

Deuxième passage :

$$u = 2 \times 11 - 3 = 19.$$

Ainsi,

$$u \leftarrow 7 \longrightarrow 11 \longrightarrow 19.$$

**Conclusion.** La valeur affichée est 19.

## Exercice 4

REP

**Énoncé** Dire si chaque boucle est **bornée** ou **non bornée**.

```
1 for i in range(5):  
2     print(i)
```

```
1 while u < 100:  
2     u = 2*u
```

**Méthode** Une boucle **for** avec un **range** possède un nombre de passages déterminé à l'avance. Une boucle **while** dépend d'une condition et son nombre de passages n'est pas fixé au départ.

Dans

```
1 for i in range(5):  
2     print(i)
```

la boucle effectue exactement 5 passages. Elle est **bornée**.

Dans

```
1 while u < 100:  
2     u = 2*u
```

le nombre de passages dépend de la valeur initiale de  $u$ . Elle est **non bornée**.

**Conclusion.** La boucle **for** est bornée ; la boucle **while** est non bornée.

**Exercice 5**

CAL, REP

**Énoncé** Compléter la trace d'exécution.

```

1 u = 3
2 for i in range(4):
3     u = u + 5
4 print(u)

```

| étape            | valeur de $u$ |
|------------------|---------------|
| départ           | 3             |
| après 1 passage  |               |
| après 2 passages |               |
| après 3 passages |               |
| après 4 passages |               |

**Méthode** La boucle est exécutée quatre fois. À chaque passage, on ajoute 5 à la valeur courante de  $u$ .

| étape            | $u$ |
|------------------|-----|
| départ           | 3   |
| après 1 passage  | 8   |
| après 2 passages | 13  |
| après 3 passages | 18  |
| après 4 passages | 23  |

Le programme affiche donc

23.

**Conclusion.** La trace est 3, 8, 13, 18, 23 et la valeur affichée est 23.

**Exercice 6**

CAL, REP

**Énoncé** Que renvoie la fonction suivante ?

```
1 def f(x):  
2     return 3*x + 2
```

Calculer mentalement :

$$f(0), \quad f(4), \quad f(-1).$$

**Méthode** Une fonction Python remplace son paramètre  $x$  par la valeur fournie lors de l'appel, puis renvoie le résultat de l'expression.

$$f(0) = 3 \times 0 + 2 = 2.$$

$$f(4) = 3 \times 4 + 2 = 14.$$

$$f(-1) = 3 \times (-1) + 2 = -1.$$

**Conclusion.**  $f(0) = 2, \quad f(4) = 14, \quad f(-1) = -1$ .

**Exercice 7**

COM

**Énoncé** Expliquer en une phrase la différence entre :

$$n = n + 1 \quad \text{et} \quad n == n + 1.$$

**Méthode** On distingue une instruction qui modifie une variable d'une expression logique qui renvoie vrai ou faux.

$$n = n + 1$$

est une **affectation** : on remplace la valeur de  $n$  par son ancienne valeur augmentée de 1.

En revanche,

$$n == n + 1$$

est un **test d'égalité** : Python vérifie si  $n$  est égal à  $n + 1$ . Pour un nombre ordinaire, ce test est faux.

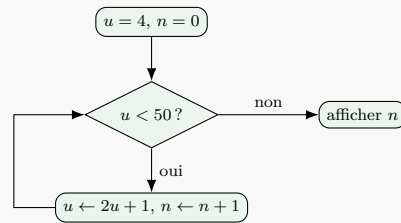
**Conclusion.** `=` modifie la variable ; `==` teste une égalité sans modifier la variable.



## Exercice 8

REP, CAL

**Énoncé** On donne le schéma de boucle suivant.



Calculer les valeurs successives de  $u$ , puis la valeur affichée.

**Méthode** On suit simultanément  $u$  et  $n$ . La boucle continue tant que  $u < 50$ .

Au départ,

$$u = 4, \quad n = 0.$$

Les passages donnent :

$$(u, n) = (9, 1),$$

$$(u, n) = (19, 2),$$

$$(u, n) = (39, 3),$$

$$(u, n) = (79, 4).$$

À ce moment,

$$79 < 50$$

est faux. La boucle s'arrête et affiche

$$n = 4.$$

**Erreur type / vigilance** La condition est testée avant chaque nouveau passage. La valeur 79 est donc calculée lors du dernier passage, puis provoque l'arrêt au test suivant.

**Conclusion.** Les valeurs de  $u$  sont 4, 9, 19, 39, 79, et la valeur affichée est 4.

## B — Boucles, suites et seuils

**Exercice 9**

CAL, REP

**Énoncé** Le programme suivant calcule des termes d'une suite.

```
1 u = 2
2 for n in range(5):
3     u = u + 3
4 print(u)
```

1. Quelle suite est définie par ce programme ?
2. Quelle valeur est affichée ?
3. Cette valeur correspond-elle à  $u_4$  ou à  $u_5$  si  $u_0 = 2$  ?

**Méthode** La variable  $u$  contient d'abord  $u_0$ . Chaque passage ajoute 3, donc fait avancer d'un rang.

Le programme définit la suite

$$u_0 = 2, \quad u_{n+1} = u_n + 3.$$

C'est une suite arithmétique de raison 3.

La boucle effectue 5 passages :

$$u_1 = 5, \quad u_2 = 8, \quad u_3 = 11, \quad u_4 = 14, \quad u_5 = 17.$$

La valeur affichée est donc

$$17.$$

Comme la valeur initiale est  $u_0$ , cinq mises à jour conduisent à

$$u_5.$$

**Conclusion.** Suite  $u_0 = 2$ ,  $u_{n+1} = u_n + 3$  ; le programme affiche  $17 = u_5$ .

**Exercice 10**

CAL, REP

**Énoncé** Même travail avec le programme suivant.

```
1 u = 5
2 for n in range(4):
3     u = 2*u
4 print(u)
```

**Méthode** La valeur initiale est  $u_0 = 5$ . Chaque passage multiplie  $u$  par 2, donc fait avancer d'un rang dans une suite géométrique.

La suite est définie par

$$u_0 = 5, \quad u_{n+1} = 2u_n.$$

Les quatre passages donnent

$$u_1 = 10, \quad u_2 = 20, \quad u_3 = 40, \quad u_4 = 80.$$

Le programme affiche donc

$$80,$$

qui correspond à

$$u_4.$$

**Conclusion.** Suite géométrique de raison 2 ; le programme affiche  $80 = u_4$ .

**Exercice 11**

REP, CAL

**Énoncé** Compléter le programme pour afficher les dix premiers termes de la suite définie par :

$$u_0 = 7, \quad u_{n+1} = u_n + 4.$$

```
1 u = ...  
2 for n in range(...):  
3     print(...)  
4     u = ...
```

**Méthode** Pour afficher les dix premiers termes  $u_0, \dots, u_9$ , on effectue dix passages. On affiche le terme courant *avant* de le remplacer par le suivant.

Un programme correct est :

```
1 u = 7  
2 for n in range(10):  
3     print(u)  
4     u = u + 4
```

La boucle affiche successivement

$$7, 11, 15, \dots, 43.$$

**Erreur type / vigilance** Si l'on met `u = u + 4` avant `print(u)`, le premier terme affiché serait  $u_1$  et non  $u_0$ .**Conclusion.** `u = 7`, `range(10)`, `print(u)`, puis `u = u + 4`.

**Exercice 12**

REP, CAL

**Énoncé** Compléter le programme pour afficher  $u_0, u_1, \dots, u_8$ , où :

$$u_0 = 3, \quad u_{n+1} = 1.5u_n.$$

```
1 u = ...  
2 for n in range(...):  
3     print(n, u)  
4     u = ...
```

**Méthode** Les termes demandés vont de  $u_0$  à  $u_8$ , soit 9 termes. On affiche avant la mise à jour.

```
1 u = 3  
2 for n in range(9):  
3     print(n, u)  
4     u = 1.5*u
```

Le rang affiché par  $n$  correspond bien au terme courant  $u_n$ .

**Conclusion.** Il faut utiliser `u = 3`, `range(9)` et `u = 1.5*u`.

**Exercice 13**

REP, CAL

**Énoncé** On veut déterminer le premier rang  $n$  tel que  $u_n \geq 100$ , pour :

$$u_0 = 4, \quad u_{n+1} = u_n + 7.$$

Compléter le programme.

```

1 u = ...
2 n = ...
3 while ...:
4     u = ...
5     n = ...
6 print(n)
```

**Méthode** On cherche le premier rang pour lequel le terme courant atteint ou dépasse 100. La boucle doit donc continuer tant que  $u < 100$ .

```

1 u = 4
2 n = 0
3 while u < 100:
4     u = u + 7
5     n = n + 1
6 print(n)
```

Mathématiquement,

$$u_n = 4 + 7n.$$

Le seuil vérifie

$$4 + 7n \geq 100.$$

On obtient l'inéquation équivalente

$$7n \geq 96,$$

puis, en divisant les deux membres par  $7 > 0$ ,

$$n \geq \frac{96}{7} \simeq 13,71.$$

Le premier entier convenable est donc

$$n = 14.$$

**Conclusion.** Le programme affiche 14, premier rang tel que  $u_n \geq 100$ .

**Exercice 14**

REP, CAL

**Énoncé** On veut déterminer le premier rang  $n$  tel que  $u_n \geq 1000$ , pour :

$$u_0 = 12, \quad u_{n+1} = 1.25u_n.$$

Compléter.

```
1 u = ...
2 n = ...
3 while ...:
4     u = ...
5     n = ...
6 print(n, u)
```

**Méthode** La suite est géométrique de raison 1,25. On continue tant que le terme courant reste strictement inférieur à 1000.

```
1 u = 12
2 n = 0
3 while u < 1000:
4     u = 1.25*u
5     n = n + 1
6 print(n, u)
```

Les calculs successifs conduisent au premier dépassement pour

$$n = 20.$$

À ce rang,

$$u_{20} = 12 \times 1,25^{20} \simeq 1040,83.$$

Au rang précédent,

$$u_{19} \simeq 832,67 < 1000.$$

**Conclusion.** Le programme affiche environ 20, 1040,83.

**Exercice 15**

CAL, REP

**Énoncé** Faire une trace d'exécution du programme.

```

1 u = 10
2 n = 0
3 while u < 50:
4     u = 1.4*u
5     n = n + 1
6 print(n)

```

| $n$ | $u$ |
|-----|-----|
| 0   | 10  |
| 1   |     |
| 2   |     |
| 3   |     |
| 4   |     |
| 5   |     |

Que représente la valeur affichée ?

**Méthode** On teste la condition  $u < 50$  avant chaque passage, puis on multiplie  $u$  par 1,4 et on augmente  $n$  de 1.

La trace est :

| $n$ | $u$     |
|-----|---------|
| 0   | 10      |
| 1   | 14      |
| 2   | 19,6    |
| 3   | 27,44   |
| 4   | 38,416  |
| 5   | 53,7824 |

Pour  $n = 4$ ,

$$u = 38,416 < 50,$$

donc un dernier passage est effectué.

Pour  $n = 5$ ,

$$u = 53,7824 \geq 50,$$

donc la boucle s'arrête.

La valeur affichée est

5.

Elle représente le premier rang pour lequel le terme de la suite atteint ou dépasse 50.

**Conclusion.** Le programme affiche 5.



**Exercice 16**

CAL, REP

**Énoncé** On considère le programme suivant.

```
1 S = 0
2 for k in range(1, 11):
3     S = S + k
4 print(S)
```

1. Quelle somme est calculée ?
2. Quelle valeur est affichée ?
3. Modifier mentalement le programme pour calculer  $1 + 2 + \dots + 100$ .

**Méthode** Dans `range(1,11)`, les valeurs prises par  $k$  sont  $1, 2, \dots, 10$ . La variable  $S$  accumule ces valeurs.

Le programme calcule

$$1 + 2 + \dots + 10.$$

Ainsi

$$S = \frac{10 \times 11}{2} = 55.$$

La valeur affichée est donc

$$55.$$

Pour calculer

$$1 + 2 + \dots + 100,$$

il suffit de remplacer la ligne de boucle par

```
1 for k in range(1, 101):
```

**Conclusion.** Somme calculée :  $1 + \dots + 10$ ; valeur affichée `55`; pour aller jusqu'à 100, utiliser `range(1,101)`.

**Exercice 17**

REP, CAL

**Énoncé** Compléter le programme pour calculer :

$$3 + 6 + 12 + 24 + \dots + 3 \times 2^{10}.$$

```
1 u = ...
2 S = ...
3 for n in range(...):
4     u = ...
5     S = ...
6 print(S)
```

**Méthode** Le premier terme 3 doit être inclus dans la somme, puis on effectue dix multiplications par 2 pour aller jusqu'au terme  $3 \times 2^{10}$ .

Un programme correct est :

```
1 u = 3
2 S = 3
3 for n in range(10):
4     u = 2*u
5     S = S + u
6 print(S)
```

Il calcule

$$3 + 6 + 12 + \dots + 3 \times 2^{10}.$$

La somme vaut

$$3(1 + 2 + \dots + 2^{10}) = 3(2^{11} - 1) = 6141.$$

**Conclusion.** Le programme affiche 6141.

## Exercice 18

RAI, COM

**Énoncé** Un élève propose le programme suivant pour trouver le premier rang tel que  $u_n \geq 100$ , avec  $u_0 = 5$  et  $u_{n+1} = u_n + 6$ .

```

1 u = 5
2 n = 0
3 while u < 100:
4     n = n + 1
5 print(n)

```

1. Pourquoi ce programme ne convient-il pas ?
2. Quelle ligne faut-il ajouter ?
3. Où faut-il l'ajouter ?

**Méthode** La condition de la boucle porte sur  $u$ . Il faut donc modifier  $u$  à l'intérieur de la boucle ; sinon la condition reste toujours vraie.

Dans le programme proposé,

$$u = 5$$

n'est jamais modifié. Comme

$$5 < 100,$$

la condition  $u < 100$  reste vraie indéfiniment : la boucle ne s'arrête pas.

Il faut ajouter la ligne

```

1 u = u + 6

```

à l'intérieur de la boucle **while**, par exemple avant ou après l'incréméntation de  $n$  :

```

1 u = 5
2 n = 0
3 while u < 100:
4     u = u + 6
5     n = n + 1
6 print(n)

```

Le premier rang obtenu est

$$n = 16,$$

car

$$u_{15} = 95 < 100$$

et

$$u_{16} = 101 \geq 100.$$

**Erreur type / vigilance** Une boucle **while** risque d'être infinie si aucune instruction ne fait évoluer les variables qui interviennent dans sa condition.

**Conclusion.** La ligne manquante est `u = u + 6`, à placer dans la boucle ; le programme corrigé affiche `16`.

**Exercice 19**

REP, CAL

**Énoncé** On considère la fonction suivante.

```
1 def terme_arith(u0, r, n):  
2     u = u0  
3     for k in range(n):  
4         u = u + r  
5     return u
```

1. Calculer à la main `terme_arith(4,3,5)`.
2. Quelle formule mathématique cette fonction utilise-t-elle implicitement ?

**Méthode** La fonction part de  $u_0$ , puis effectue exactement  $n$  ajouts de la raison  $r$ .

Pour

$$\text{terme\_arith}(4,3,5),$$

on part de

$$u = 4.$$

Les cinq passages donnent :

$$7, 10, 13, 16, 19.$$

La fonction renvoie donc

$$19.$$

Mathématiquement, elle calcule implicitement

$$u_n = u_0 + nr.$$

Ici,

$$u_5 = 4 + 5 \times 3 = 19.$$

**Conclusion.**  $\text{terme\_arith}(4,3,5) = 19$ , et la formule implicite est  $u_n = u_0 + nr$ .

**Exercice 20**

REP, CAL

**Énoncé** Écrire une fonction `terme_geom(u0,q,n)` qui renvoie le terme  $u_n$  de la suite géométrique définie par :

$$u_0 = u0, \quad u_{n+1} = qu_n.$$

**Méthode** On reproduit la même structure que pour une suite arithmétique, mais la mise à jour devient une multiplication par  $q$ .

Une fonction possible est :

```
1 def terme_geom(u0, q, n):
2     u = u0
3     for k in range(n):
4         u = q*u
5     return u
```

Après  $n$  passages, la variable  $u$  contient

$$u_n = u_0q^n.$$

**Conclusion.** La fonction ci-dessus renvoie bien le terme  $\boxed{u_n}$  de la suite géométrique.

## Exercice 21

REP, CAL

**Énoncé** On considère la fonction suivante.

```

1 def seuil(u0, q, A):
2     u = u0
3     n = 0
4     while u < A:
5         u = q*u
6         n = n + 1
7     return n

```

1. Que renvoie `seuil(10,2,100)` ?
2. Interpréter le résultat en termes de suite.
3. Que se passe-t-il si  $q = 1$  et  $u_0 < A$  ?

**Méthode** On suit les valeurs 10, 20, 40, ... jusqu'au premier terme qui atteint ou dépasse 100.

Avec

$$u_0 = 10 \quad \text{et} \quad q = 2,$$

on obtient

$$u_1 = 20, \quad u_2 = 40, \quad u_3 = 80, \quad u_4 = 160.$$

Le premier terme supérieur ou égal à 100 est donc  $u_4$ . Ainsi

$$\text{seuil}(10, 2, 100)$$

renvoie

$$\boxed{4}.$$

Ce résultat est le premier rang  $n$  tel que

$$u_n \geq 100.$$

Si

$$q = 1$$

et

$$u_0 < A,$$

alors la mise à jour

$$u = q u$$

ne change jamais  $u$ . La condition  $u < A$  reste vraie indéfiniment : la boucle ne s'arrête pas.

**Erreur type / vigilance** Un algorithme de seuil ne termine pas automatiquement pour toutes les valeurs des paramètres. Il faut que la suite puisse effectivement atteindre le seuil recherché.

**Conclusion.**  $\boxed{4}$ . Si  $q = 1$  et  $u_0 < A$ , la boucle est infinie.

**Exercice 22**

RAI, COM

**Énoncé** Un élève écrit :

```

1 def seuil(u0, r, A):
2     u = u0
3     n = 0
4     while u <= A:
5         u = u + r
6     return n

```

Repérer deux erreurs et proposer une version corrigée.

**Méthode** On contrôle deux éléments : la condition doit correspondre au seuil recherché, et le rang  $n$  doit être mis à jour à chaque nouveau terme.

Première erreur : la variable

$$n$$

n'est jamais modifiée. Même si la boucle finit, la fonction renverrait toujours 0.

Deuxième erreur : si l'on cherche le premier rang tel que

$$u_n \geq A,$$

la boucle doit continuer tant que

$$u < A,$$

et non tant que  $u \leq A$ .

Une version corrigée est :

```

1 def seuil(u0, r, A):
2     u = u0
3     n = 0
4     while u < A:
5         u = u + r
6         n = n + 1
7     return n

```

**Erreur type / vigilance** Si  $r \leq 0$  et  $u0 < A$ , cette version peut encore ne jamais atteindre le seuil. La correction répond ici au modèle d'une suite arithmétique croissante.

**Conclusion.** Il faut incrémenter  $n$  et utiliser la condition  $u < A$  pour rechercher le premier rang tel que  $u_n \geq A$ .

**Exercice 23**

REP, CAL

**Énoncé** On considère le programme suivant.

```
1 L = []
2 u = 3
3 for n in range(6):
4     L.append(u)
5     u = 2*u + 1
6 print(L)
```

1. Compléter la liste affichée.
2. Pourquoi l'instruction `L.append(u)` est-elle placée avant la mise à jour de  $u$  ?
3. Donner la valeur de `L[0]` et de `L[3]`. Quelle instruction supprimerait le dernier terme de `L` ?

**Méthode** À chaque passage, on ajoute d'abord le terme courant à la liste, puis on calcule le terme suivant.

Au départ,

$$u = 3.$$

La liste reçoit successivement :

$$3, 7, 15, 31, 63, 127.$$

Ainsi le programme affiche

$$[3, 7, 15, 31, 63, 127].$$

L'instruction

$$L.append(u)$$

est placée avant la mise à jour pour stocker d'abord le terme courant  $u_n$ . Si on la plaçait après

$$u = 2*u + 1,$$

la liste commencerait par  $u_1 = 7$  et  $u_0 = 3$  serait oublié.**Conclusion.** Liste affichée :  $[3, 7, 15, 31, 63, 127]$ . L'ajout avant la mise à jour permet d'inclure  $u_0$ .**Conclusion.** `L[0] = 3` et `L[3] = 31` ; `del L[-1]` (ou `del L[5]`) supprimerait le dernier terme de `L`.



**Exercice 24**

REP, CAL

**Énoncé** Écrire une fonction `liste_suite(u0,r,N)` qui renvoie la liste :

$$[u_0, u_1, \dots, u_N]$$

pour la suite arithmétique de premier terme  $u_0$  et de raison  $r$ .

**Méthode** La liste demandée contient les termes de  $u_0$  à  $u_N$ , soit  $N + 1$  termes. On ajoute le terme courant puis on passe au suivant.

Une fonction possible est :

```
1 def liste_suite(u0, r, N):  
2     L = []  
3     u = u0  
4     for n in range(N + 1):  
5         L.append(u)  
6         u = u + r  
7     return L
```

Par exemple,

`liste_suite(4,3,2)`

renvoie

`[4, 7, 10]`.

**Conclusion.** La fonction ci-dessus renvoie bien  $[u_0, u_1, \dots, u_N]$ .

**Exercice 25**

MOD, REP

**Énoncé** Un capital de 500 euros augmente de 4 % par an. On veut écrire une fonction qui renvoie le premier nombre d'années nécessaire pour dépasser un montant  $A$ .

1. Identifier la suite utilisée.
2. Compléter la fonction.

```

1 def attente(A):
2     C = ...
3     n = ...
4     while ...:
5         C = ...
6         n = ...
7     return n

```

**Méthode** Une hausse annuelle de 4 % signifie que le capital est multiplié chaque année par

$$1,04.$$

Comme on veut *dépasser*  $A$ , la boucle continue tant que  $C \leq A$ .

La suite de capitaux vérifie

$$C_0 = 500, \quad C_{n+1} = 1,04 C_n.$$

C'est une suite géométrique de raison

$$1,04.$$

Une fonction correcte est :

```

1 def attente(A):
2     C = 500
3     n = 0
4     while C <= A:
5         C = 1.04*C
6         n = n + 1
7     return n

```

La valeur renvoyée est le premier nombre entier d'années  $n$  pour lequel

$$C_n > A.$$

**Erreur type / vigilance** Le mot « dépasser » impose une inégalité stricte. Si l'on voulait seulement « atteindre ou dépasser »  $A$ , on utiliserait la condition `while C < A`.

**Conclusion.** Suite  $C_{n+1} = 1,04C_n$  avec  $C_0 = 500$ , et fonction complétée ci-dessus.

**Exercice 26**

REP, CAL

**Énoncé** On veut simuler un lancer de dé équilibré à six faces.

```
1 import random
2
3 def de():
4     return random.randint(1, 6)
```

1. Que peut renvoyer la fonction `de()` ?
2. Écrire un programme qui lance le dé 20 fois et stocke les résultats dans une liste.
3. Quelle instruction permet de compter le nombre de 6 obtenus dans la liste  $L$  ?

**Méthode** La fonction `random.randint(1,6)` choisit aléatoirement un entier parmi les six faces. On répète ensuite l'appel vingt fois.

La fonction

`de()`

peut renvoyer l'un des entiers

1, 2, 3, 4, 5, 6.

Un programme possible pour vingt lancers est :

```
1 L = []
2 for i in range(20):
3     L.append(de())
4 print(L)
```

Pour compter le nombre de 6 présents dans la liste  $L$ , on peut écrire :

```
1 L.count(6)
```

**Conclusion.** `de()` renvoie un entier de 1 à 6 ; vingt résultats peuvent être stockés dans une liste et `L.count(6)` compte les 6.

**Exercice 27**

MOD, REP

**Énoncé** On simule un jeu : on gagne 5 euros si le dé donne 6, sinon on perd 1 euro.

1. Compléter la fonction `gain()`.
2. Écrire une phrase décrivant la variable simulée.

```
1 import random
2
3 def gain():
4     d = random.randint(1, 6)
5     if ...:
6         return ...
7     else:
8         return ...
```

**Méthode** On simule d’abord le résultat du dé, puis on traduit la règle du jeu par une condition.

La fonction complétée est :

```
1 import random
2
3 def gain():
4     d = random.randint(1, 6)
5     if d == 6:
6         return 5
7     else:
8         return -1
```

La variable aléatoire simulée est le **gain en euros d’une partie** : elle vaut 5 euros si le dé donne 6, et  $-1$  euro sinon.

**Conclusion.** La fonction renvoie `5` en cas de 6, et `-1` sinon.

**Exercice 28**

CH, REP

**Énoncé** On veut estimer le gain moyen du jeu précédent sur 1000 parties. Compléter.

```
1 S = 0
2 for i in range(...):
3     S = S + ...
4 m = S / ...
5 print(m)
```

Expliquer pourquoi le résultat peut changer lorsqu'on relance le programme.

**Méthode** On additionne les gains de 1000 parties indépendantes, puis on divise le total par 1000.

Le programme complété est :

```
1 S = 0
2 for i in range(1000):
3     S = S + gain()
4 m = S / 1000
5 print(m)
```

Le nombre affiché est le gain moyen observé sur les 1000 simulations.

Lorsqu'on relance le programme, les lancers simulés par le module `random` ne sont généralement pas les mêmes. Le nombre de 6 change donc légèrement, et le gain moyen calculé peut changer lui aussi.**POINT FONDAMENTAL À RETENIR EN MATHÉMATIQUES** Une simulation produit une estimation numérique dépendant de l'échantillon aléatoire obtenu. Augmenter le nombre de répétitions tend à stabiliser l'estimation, sans rendre deux exécutions nécessairement identiques.**Conclusion.** Il faut utiliser `range(1000)`, ajouter `gain()` à  $S$ , puis calculer `S/1000`.**D — Problèmes et synthèse**

**Exercice 29**

MOD, CAL, REP

**Énoncé** Une espèce invasive couvre  $12 \text{ m}^2$  d'un lac au départ. Chaque semaine, la surface couverte augmente de 35 %. On veut savoir au bout de combien de semaines elle dépassera  $500 \text{ m}^2$ .

1. Écrire la relation de récurrence vérifiée par la suite.
2. Compléter l'algorithme en langage naturel.
3. Écrire ensuite un programme Python.

```

u ← 12
n ← 0
Tant que .....
    u ← .....
    n ← .....
Fin Tant que
Afficher .....
```

**Méthode** Une hausse hebdomadaire de 35 % signifie que la surface est multipliée par

1,35.

On cherche le premier rang où la surface *dépasse* 500, donc la boucle continue tant que  $u \leq 500$ .

La suite vérifie

$$u_0 = 12, \quad u_{n+1} = 1,35u_n.$$

L'algorithme en langage naturel est :

```

u ← 12
n ← 0
Tant que  $u \leq 500$ 
    u ← 1,35u
    n ← n + 1
Fin Tant que
Afficher n
```

En Python :

```

1 u = 12
2 n = 0
3 while u <= 500:
4     u = 1.35*u
5     n = n + 1
6 print(n)
```

Le programme s'arrête pour

$$n = 13.$$

À ce rang,

$$u_{13} \simeq 593,64 > 500.$$

**Conclusion.** La surface dépasse  $500 \text{ m}^2$  pour la première fois après 13 semaines.

## Exercice 30

MOD, RAI

**Énoncé** Une association reçoit 200 euros de dons le premier mois. Chaque mois, les dons augmentent de 15 euros. Elle veut savoir au bout de combien de mois le total cumulé dépassera 3000 euros.

1. Faut-il comparer un terme ou une somme à 3000 ?
2. Compléter le programme.

```

1 u = 200
2 S = ...
3 n = 0
4 while ...:
5     n = ...
6     u = ...
7     S = ...
8 print(n, S)

```

**Méthode** La question porte sur le **total cumulé** des dons, donc on doit comparer la variable  $S$  à 3000, et non le terme mensuel  $u$ .

On initialise

$$u = 200$$

pour le premier mois et

$$S = 200$$

pour le cumul après ce premier mois.

Comme on cherche à *dépasser* 3000, on continue tant que

$$S \leq 3000.$$

Un programme correct est :

```

1 u = 200
2 S = 200
3 n = 0
4 while S <= 3000:
5     n = n + 1
6     u = u + 15
7     S = S + u
8 print(n, S)

```

Le programme affiche

$$n = 10 \quad \text{et} \quad S = 3025.$$

Le rang  $n = 0$  correspond au premier mois. Le rang 10 correspond donc au

$$10 + 1 = 11^{\text{e}}$$

mois.

**Erreur type / vigilance** Ici, la valeur affichée  $n = 10$  est un **rang**, pas directement un nombre de mois écoulés depuis le début de la liste. Le cumul  $S$  contient alors onze termes, de  $u_0$  à  $u_{10}$ .

**Conclusion.** Il faut comparer le cumul  $S$  à 3000. Le seuil est dépassé au 11<sup>e</sup> mois, avec 3025 € cumulés.

## Exercice 31

CH, RAI, COM

**Énoncé** Un élève écrit le programme suivant pour calculer la somme des termes  $u_0 + \dots + u_{10}$ , avec  $u_0 = 4$  et  $u_{n+1} = 3u_n$ .

```

1 u = 4
2 S = 0
3 for n in range(10):
4     u = 3*u
5     S = S + u
6 print(S)
```

1. Quelle somme ce programme calcule-t-il réellement ?
2. Pourquoi  $u_0$  n'est-il pas pris en compte ?
3. Proposer une correction.

**Méthode** On observe l'ordre des deux instructions dans la boucle : le programme calcule d'abord le terme suivant, puis l'ajoute à  $S$ .

Au départ,

$$u = u_0 = 4 \quad \text{et} \quad S = 0.$$

Au premier passage,

$$u = 3u_0 = u_1,$$

puis  $u_1$  est ajouté à  $S$ .

Après dix passages, le programme a donc additionné

$$u_1 + u_2 + \dots + u_{10},$$

et non

$$u_0 + u_1 + \dots + u_{10}.$$

Le terme  $u_0$  n'est jamais ajouté car la mise à jour

$$u = 3*u$$

est exécutée avant

$$S = S + u.$$

Une correction simple consiste à initialiser  $S$  avec  $u_0$  :

```

1 u = 4
2 S = 4
3 for n in range(10):
4     u = 3*u
5     S = S + u
6 print(S)
```

On obtient alors

$$u_0 + \dots + u_{10} = 354292.$$

Le programme initial calculait

$$u_1 + \dots + u_{10} = 354288.$$

**Conclusion.** Le programme initial oublie  $u_0$ . Une correction est `S = 4` avant la boucle.



**Exercice 32**

MOD, REP

**Énoncé** On étudie la suite définie par :

$$u_0 = 100, \quad u_{n+1} = 0,85u_n + 12.$$

1. Cette suite est-elle arithmétique ? géométrique ? Justifier.
2. Écrire une fonction `terme(n)` qui renvoie  $u_n$ .
3. Écrire une fonction `liste(N)` qui renvoie la liste des termes de  $u_0$  à  $u_N$ .
4. À quoi peut servir une liste de termes dans l'étude d'une suite ?

**Méthode** On teste d'abord la nature de la suite, puis on traduit la récurrence par une boucle qui effectue  $n$  mises à jour.

La relation est

$$u_{n+1} = 0,85u_n + 12.$$

La différence vaut

$$u_{n+1} - u_n = -0,15u_n + 12,$$

qui dépend de  $u_n$  : elle n'est pas constante en général. La suite n'est donc pas arithmétique.De même, lorsque  $u_n \neq 0$ ,

$$\frac{u_{n+1}}{u_n} = 0,85 + \frac{12}{u_n},$$

qui dépend de  $u_n$ . La suite n'est donc pas géométrique.Une fonction qui renvoie  $u_n$  est :

```

1 def terme(n):
2     u = 100
3     for k in range(n):
4         u = 0.85*u + 12
5     return u

```

Une fonction qui renvoie les termes de  $u_0$  à  $u_N$  est :

```

1 def liste(N):
2     L = []
3     u = 100
4     for n in range(N + 1):
5         L.append(u)
6         u = 0.85*u + 12
7     return L

```

Une liste de termes permet notamment de vérifier des calculs, observer numériquement une évolution, construire un graphique et formuler des conjectures sur le comportement de la suite.

**Conclusion.** La suite n'est ni arithmétique ni géométrique ; les deux fonctions ci-dessus calculent un terme ou une liste de termes.

**Exercice 33**

CH, MOD, REP

**Énoncé** On veut conjecturer le comportement de la suite :

$$u_0 = 2, \quad u_{n+1} = \frac{1}{2}u_n + 3.$$

1. Compléter le programme pour afficher les 20 premiers termes.
2. À l'aide des valeurs affichées, formuler une conjecture.

```
1 u = ...  
2 for n in range(...):  
3     print(n, u)  
4     u = ...
```

**Méthode** On affiche le terme courant avant de calculer le suivant. Vingt passages permettent d'afficher  $u_0, \dots, u_{19}$ .

Le programme complété est :

```
1 u = 2  
2 for n in range(20):  
3     print(n, u)  
4     u = 0.5*u + 3
```

Les premières valeurs sont

2, 4, 5, 5,5, 5,75, 5,875,...

Plus loin, les valeurs sont très proches de

6.

On peut donc conjecturer que la suite est croissante et que ses termes se rapprochent de 6.

**Erreur type / vigilance** Le programme permet ici de **conjecturer** un comportement à partir de valeurs numériques. L'affichage de vingt termes ne constitue pas à lui seul une démonstration valable pour tous les rangs.

**Conclusion.** Les valeurs suggèrent que  $(u_n)$  augmente et se rapproche de 6.

## Exercice 34

CH, MOD, RAI

**Énoncé** Une urne contient 3 boules rouges et 7 boules bleues. On tire une boule au hasard. Si elle est rouge, on gagne 4 euros ; sinon on perd 2 euros.

1. Compléter la fonction `gain()` qui simule un tirage.
2. Compléter ensuite le programme qui simule 500 tirages et affiche le gain moyen.
3. Que devrait-on calculer en probabilités pour contrôler ce résultat ?

```

1 import random
2
3 def gain():
4     x = random.random()
5     if x < ...:
6         return ...
7     else:
8         return ...
9
10 S = 0
11 for i in range(...):
12     S = ...
13 print(...)
```

**Méthode** La probabilité d'obtenir une boule rouge est

$$\frac{3}{10} = 0,3.$$

La condition `x < 0.3` simule donc l'événement « tirer une boule rouge ».

La fonction de simulation peut être écrite :

```

1 import random
2
3 def gain():
4     x = random.random()
5     if x < 0.3:
6         return 4
7     else:
8         return -2
```

Pour 500 tirages :

```

1 S = 0
2 for i in range(500):
3     S = S + gain()
4 print(S / 500)
```

Le nombre affiché est le gain moyen observé sur les 500 tirages simulés.

Pour contrôler ce résultat par un calcul de probabilités, on calcule le gain moyen théorique :

$$4 \times \frac{3}{10} + (-2) \times \frac{7}{10}.$$

Ainsi

$$1,2 - 1,4 = -0,2.$$

Le gain moyen simulé devrait donc être proche de

$$-0,2 \text{ €}$$

lorsque le nombre de simulations est suffisamment grand.

**Conclusion.** La condition est  $x < 0.3$ , les gains sont 4 et  $-2$ , et le gain moyen théorique vaut  $-0,2$  €.

**Exercice 35**

CH, COM

**Énoncé** Bilan de méthode. Rédiger en cinq lignes maximum une fiche expliquant comment lire un programme, faire une trace, puis interpréter la valeur affichée. Faire apparaître les mots : *affectation*, *boucle*, *condition*, *seuil*, *variable*.

**Méthode** La fiche doit suivre l'ordre réel d'exécution d'un programme : valeurs initiales, répétitions, test d'arrêt, puis interprétation.

Une **affectation** donne ou modifie la valeur d'une **variable**.

Pour faire une trace, on note les valeurs des variables après chaque passage dans une **boucle**.

Avec **while**, on vérifie à chaque étape la **condition** et la manière dont elle peut devenir fausse.

Dans un problème de **seuil**, on repère le premier passage où la valeur recherchée atteint ou dépasse la borne fixée.

Enfin, on traduit la valeur affichée dans le contexte : rang, nombre d'étapes, terme, cumul ou moyenne simulée.

**Conclusion.** La méthode de lecture repose sur l'état des variables, l'exécution de la boucle, la condition d'arrêt et l'interprétation finale.